

Ethics and Legality Integration into the Agile Software Development Pipeline

From Agile Theory to Compliance-as-Code

Social and Ethical Issues

Contents

1	Introduction	1
2	Conceptual Foundations	1
2.1	Cybernetics: feedback as the organising principle	1
2.2	Machine intelligence and the limits of behavioural imitation	1
2.3	Syntax, semantics, and the Chinese Room	2
2.4	Responsibility, many hands, and the responsibility gap	2
3	The Agile-VSD model	2
3.1	The Agile sprint loop	2
3.2	Expansion with Value Sensitive Design (VSD)	2
3.3	Stakeholder tokens and value source analysis	3
3.4	Connection to responsibility	4
4	Conceptual Engineering and the First Value Hierarchy	4
4.1	Why conceptual engineering is needed	4
4.2	Conceptual Engineering inside sprint planning	4
4.3	First value hierarchy	4
4.4	Integration with the Agile-VSD loop	5
5	Requirements Engineering through GORE and SLEEC	5
5.1	From values to normative goals	5
5.2	GORE artefacts	5
5.3	From normative goals to SLEEC rules	6
5.4	Testability and conflict detection	6
5.5	Integration with the Agile-VSD framework	6
6	DevOps and Compliance-as-Code	6
6.1	From ethical requirements to pipeline controls	6
6.2	Integration with Agile-VSD and GORE-SLEEC	7
6.3	Shift-left compliance	7
6.4	Policy-as-Code tools	8
6.5	Active and passive responsibility in DevOps	9
7	Expansion 3: MLOps and Runtime Assurance	9
7.1	Why AI requires a further expansion	9
7.2	The W-shaped AI lifecycle	9
7.3	Design-time assurance	9
7.4	Runtime assurance	10
7.5	Meaningful Human Control	10
7.6	Connection to superintelligence and value alignment	10

8 Integrated Final Pipeline	11
8.1 Final layered model	11
8.2 Traceability matrix	11
9 Mapping Course Topics to the Pipeline	12
10 Implementation Blueprint	12
11 Conclusion	13

1 Introduction

Modern software systems increasingly operate as socio-technical infrastructures: they do not merely execute functions, but mediate access to services, process personal data, automate decisions, and shape organisational behaviour. For this reason, a software pipeline cannot be evaluated only in terms of functional correctness, delivery speed, or security. It must also make ethical values and legal obligations explicit, traceable, testable, and monitorable.

This report proposes a progressive engineering model for integrating ethics and legality into a contemporary software development pipeline. The model starts from an Agile sprint loop and expands it through four layers:

1. **Agile based on Value Sensitive Design (VSD)** [5]: values and stakeholders enter the backlog.
2. **Conceptual Engineering with value hierarchy** [6]: ambiguous values are clarified before becoming requirements.
3. **GORE + SLEEC** [7]: values become normative goals and then formal, testable rules.
4. **DevOps/MLOps + Compliance-as-Code + Runtime Assurance** [13, 10, 9, 8, 11]: ethical and legal requirements are enforced through CI/CD gates, policy engines, monitoring, and human oversight.

The central thesis is that responsible software engineering requires a shift from *late ethical review* to *continuous ethical and legal assurance*. This also reframes responsibility into two dimensions: **Active responsibility** (forward-looking): the engineering organisation designs, tests, monitors, and documents systems to prevent foreseeable harm; **Passive responsibility** (backward-looking): when failures occur, the organisation must reconstruct who decided what, which evidence was available, which controls were applied, and why the deployed system behaved as it did.

2 Conceptual Foundations

2.1 Cybernetics: feedback as the organising principle

The proposed pipeline is cybernetic in structure [1]. It is organised around feedback loops that compare observed behaviour with desired goals and then correct the system or the process. Agile retrospectives, CI test failures, policy violations, model drift alerts, incident postmortems, and human override mechanisms all instantiate negative feedback.

This matters because ethical and legal compliance cannot be treated as a static checklist. Values evolve, stakeholders reinterpret impacts, operational environments change, and AI models degrade or behave unexpectedly. Therefore, the pipeline must repeatedly evaluate whether the system still satisfies its goals.

2.2 Machine intelligence and the limits of behavioural imitation

The Turing-test tradition frames intelligence behaviourally [2]: a machine may appear intelligent if its outputs are indistinguishable from those of a human. However, software engineering for

responsible AI cannot rely only on output imitation. A system may produce convincing behaviour while still violating privacy, amplifying bias, or hiding causal responsibility. The distinction between weak and strong AI is therefore operationally relevant: most deployed AI systems should be engineered as powerful decision-support artefacts, not as autonomous moral subjects.

2.3 Syntax, semantics, and the Chinese Room

Searle’s Chinese Room argument is useful as a warning for formal compliance [3]. A rule engine may manipulate symbols correctly while lacking semantic understanding of the underlying value. This is not an argument against formalisation; rather, it shows why formal rules must remain connected to stakeholder interpretation, domain expertise, legal review, and human oversight. In the proposed pipeline, SLEEC rules and policy-as-code are therefore treated as *engineering artefacts requiring governance*, not as replacements for judgement.

2.4 Responsibility, many hands, and the responsibility gap

Modern pipelines distribute action across product owners, developers, data engineers, security engineers, ML engineers, legal teams, platform teams, policy engines, and runtime systems. This generates the classic problem of “many hands”: outcomes are collectively produced, but accountability can become diffuse. In AI systems, this may create a responsibility gap, especially when decisions emerge from data-dependent models and adaptive behaviours [4].

The pipeline proposed in this report addresses this by assigning responsibility at each level:

- **Requirements level:** who accepted a value definition and a normative goal?
- **Implementation level:** who implemented the technical control?
- **CI/CD level:** which automated gate passed or failed?
- **Runtime level:** which monitor detected or missed a deviation?
- **Governance level:** who approved exceptions, overrides, or residual risks?

3 The Agile-VSD model

3.1 The Agile sprint loop

The initial diagram is a standard Agile loop. Its strength is rapid iteration; its weakness is that values may remain implicit unless they are deliberately introduced into sprint artefacts.

3.2 Expansion with Value Sensitive Design (VSD)

Value Sensitive Design is useful here because it is not an external ethical audit added after implementation. It is a design approach that incorporates human values throughout design work. The Agile-VSD literature argues that VSD can be integrated into existing Agile workflows rather than replacing them; values can be represented and refined inside sprint planning, execution, evaluation, and backlog management [5].

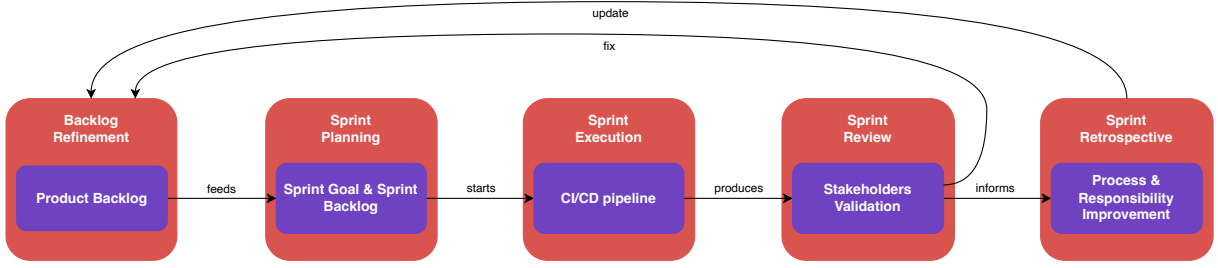


Figure 1: Baseline Agile/Scrum sprint loop. The loop connects backlog refinement, sprint planning, sprint execution, sprint review, and sprint retrospective through delivery and feedback flows.

How the diagram expands. The Agile loop in Figure 1 gains a value-oriented layer around every normal sprint activities:

- **Planning:** identify direct and indirect stakeholders; elicit values; detect value tensions.
- **Backlog refinement:** translate values into epics, user stories, acceptance criteria, and explicit constraints.
- **Development:** implement design decisions that support values such as privacy, fairness, and accessibility.
- **Testing:** validate the system also from a value-sensitive perspective.
- **Review/retrospective:** collect stakeholder feedback, document ethical debt, and update value assumptions.

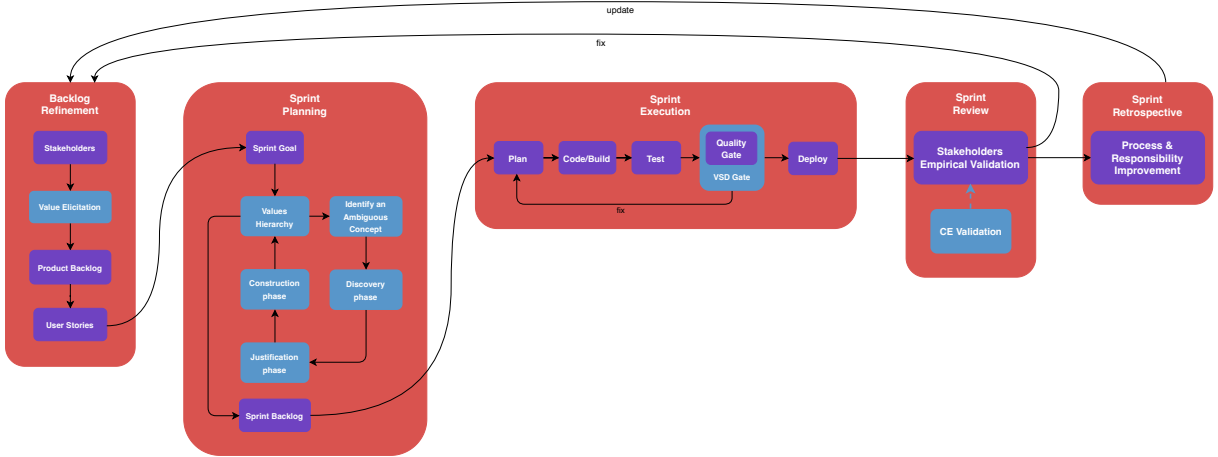


Figure 2: Agile/Scrum sprint loop enriched by a Value Sensitive Design layer. VSD activities feed the backlog, planning, execution, and review points instead of being treated as a separate downstream phase.

3.3 Stakeholder tokens and value source analysis

Two VSD methods are especially useful in a software pipeline:

Stakeholder tokens Physical or digital representations of affected actors. They force the team to keep visible not only primary users, but also data subjects, maintainers, regulators, vulnerable groups, non-users, and indirectly affected communities.

Value source analysis A structured identification of where values come from: stakeholders, law, organisational mission, professional codes, domain standards, social expectations, and risk assessments.

VSD element	Agile artefact	Engineering output
Stakeholder token	Persona / backlog actor	Explicit stakeholder impact record
Value source	Epic or value constraint	Source-traceable value statement
Value tension	Risk item / ethical debt	Trade-off requiring review
Value hierarchy	Backlog structure	Value $\rightarrow$ norm $\rightarrow$ requirement chain

Table 1: VSD inputs translated into Agile artefacts.

3.4 Connection to responsibility

At this stage, **active responsibility** means preventing ethical blind spots before implementation. The team actively asks who is affected, what values matter, and which harms are foreseeable. **Passive responsibility** begins with documentation: stakeholder assumptions and trade-offs must be recorded so that later decisions can be reconstructed.

4 Conceptual Engineering and the First Value Hierarchy

4.1 Why conceptual engineering is needed

A value such as privacy, autonomy, transparency, or fairness is too abstract to be directly implemented; so, different conceptualisations of the same value can imply different technical requirements. The Design for Values and Conceptual Engineering literature argues that value-based requirements engineering needs systematic conceptual work because different conceptions of abstract values lead to different design requirements [6].

4.2 Conceptual Engineering inside sprint planning

During planning, the team should run a short conceptual clarification step:

1. **Discovery:** how is the value currently understood by stakeholders and in the domain?
2. **Justification:** does the current concept serve the intended moral, legal, and engineering function?
3. **Construction:** which refined conception should be adopted for this system?

4.3 First value hierarchy

The first stable artefact is a value hierarchy:

Value $\rightarrow$ Norm $\rightarrow$ Design requirement $\rightarrow$ Verification artefact

Value	Norm	Requirement	Verification
Privacy	Data minimisation	Collect only fields required for the declared purpose.	Schema test; privacy review
Autonomy	Meaningful user control	Provide users with the ability to change their choices or appeal decisions.	UX test; audit log
Fairness	Non-discrimination	Evaluate model performance across relevant groups.	Fairness test report
Transparency	Explainability	Provide understandable reasons for consequential decisions.	Explanation acceptance test
Security	Integrity and confidentiality	Encrypt personal data at rest and in transit.	IaC/policy test

Table 2: Some examples of value hierarchy.

For GDPR-relevant systems, this hierarchy should be aligned with Article 5 principles such as lawfulness, fairness, transparency, purpose limitation, data minimisation, accuracy, storage limitation, integrity/confidentiality, and accountability, and with Article 25 data protection by design and by default [15, 16, 17]. For AI systems, it should also be aligned with the AI Act’s risk-based framework and requirements such as human oversight for high-risk AI systems [18, 19].

4.4 Integration with the Agile-VSD loop

The correct role of Conceptual Engineering is not simply to “define values” once. It is a dynamic approach for assessing, refining, and possibly re-engineering the concepts used in requirements. This should be performed continuously when the system, stakeholder context, or regulatory environment changes.

5 Requirements Engineering through GORE and SLEEC

5.1 From values to normative goals

The next expansion of the model moves from value analysis to requirements engineering. Goal-Oriented Requirements Engineering (GORE) is used to represent stakeholder intentions as goals. In the ethics-aware adaptation literature, ethical values are represented as **normative goals** aligned with functional and adaptation goals, enabling analysis of trade-offs and conflicts [7].

Value → Normative goal → Functional/adaptation goal → Task → Rule

5.2 GORE artefacts

A practical GORE model for this report should distinguish:

- **Normative goals:** obligations grounded in values, law, or institutional principles.
- **Functional goals:** services the system must provide.
- **Adaptation goals:** how the system changes behaviour under context changes.

- **Obstacles:** conditions that prevent goal satisfaction.
- **Trade-offs:** when there are conflicts between goals (e.g. privacy versus safety).

5.3 From normative goals to SLEEC rules

SLEEC rules translate normative goals into a form that is close to natural language but structured enough for automated analysis. In the cited SLEEC work, rules have the general form "when <condition> then <response> [within <time>] [unless <exception>]" [7].

Listing 1: Example SLEEC-style rule for GDPR deletion.

```
rule delete_personal_data:
  when UserRequestsErasure and requestIsValid
  then DeletePersonalData within 30 days
  unless legalRetentionObligation
  then RestrictProcessing and NotifyUser within 7 days
```

Listing 2: Example SLEEC-style rule for human oversight.

```
rule human_review_high_risk_decision:
  when HighRiskAIDecisionGenerated and confidence < threshold
  then EscalateToHumanReviewer within 5 minutes
```

5.4 Testability and conflict detection

SLEEC-style rules improve engineering quality because they support:

- well-formedness checking;
- conflict detection between rules;
- mapping to acceptance tests;
- mapping to runtime monitors;
- review by non-technical stakeholders.

This stage is where values become technically actionable. However, the Chinese Room lesson still applies: formal rules must remain explainable and reviewable, because syntactic rule execution does not guarantee semantic adequacy.

5.5 Integration with the Agile-VSD framework

The Agile-VSD diagram gains a requirements refinement branch:

6 DevOps and Compliance-as-Code

6.1 From ethical requirements to pipeline controls

The next expansion shifts the focus from requirements specification to the CI/CD pipeline. Compliance-as-Code (CaC) expresses legal and organisational compliance requirements as executable, version-controlled, testable policy artefacts. The CaC literature describes this as a shift

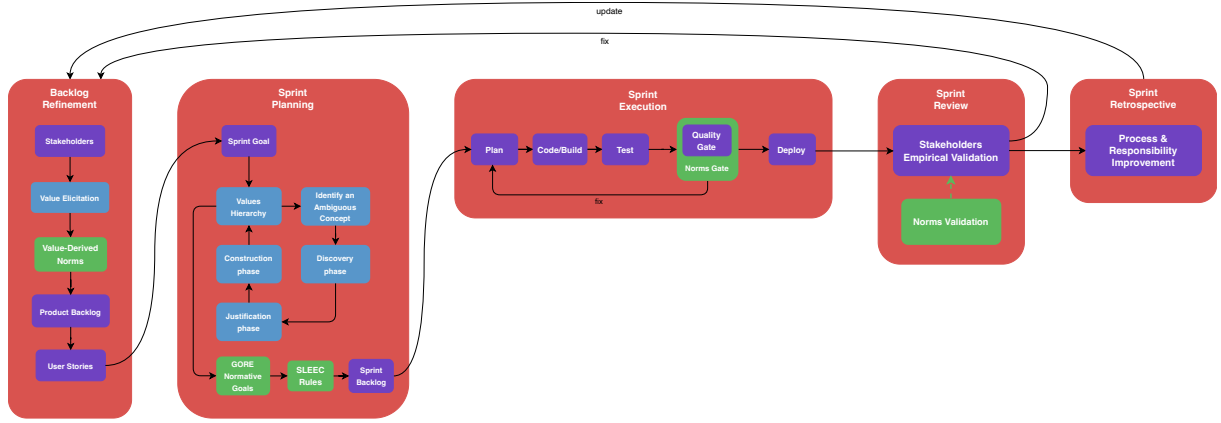


Figure 3: Detailed Agile-VSD planning view with Conceptual Engineering, GORE normative goals, SLEEC rules, and sprint backlog formation.

from document-centric and retrospective compliance toward programmatic controls embedded directly in the software delivery lifecycle [10, 9].

6.2 Integration with Agile-VSD and GORE-SLEEC

The previous requirements branch is connected to CI/CD gates:

Normative goal → SLEEC rule → Policy-as-code → CI/CD gate → Audit evidence

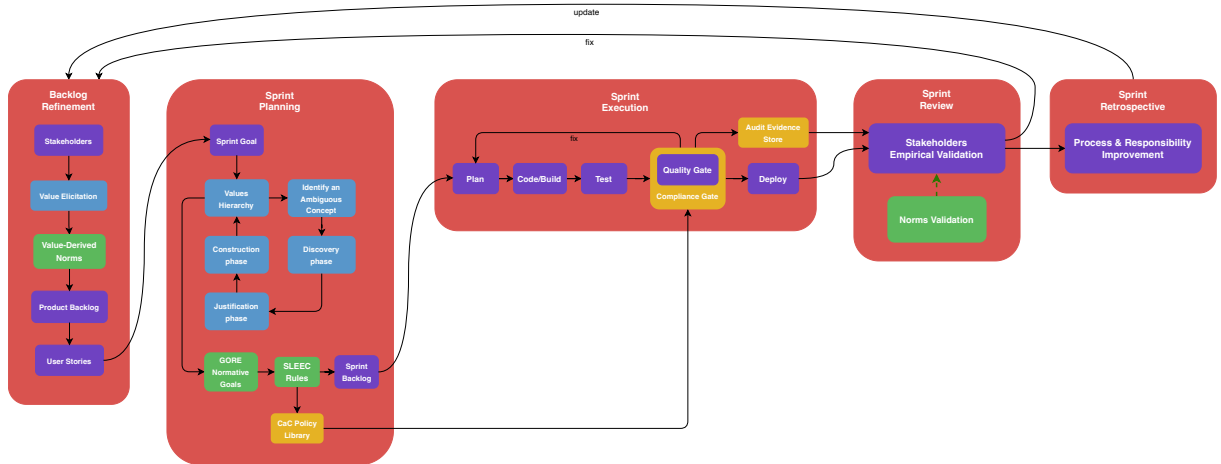


Figure 4: CI/CD expansion with Compliance-as-Code gates and audit evidence. Compliance policy evaluation is placed inside the build-test-deploy flow, and audit evidence is stored as part of the delivery process.

6.3 Shift-left compliance

Shift-left compliance means that checks are executed as early as possible, ideally before code reaches production. CaC should be integrated at multiple points:

Table 3: Compliance controls across the CI/CD pipeline.

Pipeline stage	Typical controls	Evidence produced
Code / pre-commit	secret detection, linting, privacy annotations, prohibited API checks	commit metadata, static-analysis report
Build	dependency scanning, licence checks, reproducible build	SBOM, dependency report, build signature
Test	privacy tests, fairness tests, security tests, SLEEC-derived acceptance tests	test report, model-card update, risk record
Release gate	OPA/Sentinel policy evaluation, approval workflow, exception handling	policy decision log, approver identity
Deploy	infrastructure-as-code validation, RBAC, encryption, network controls	deployment manifest, IaC compliance report
Operate	runtime monitoring, audit logs, drift detection, incident alerts	immutable logs, alert history, incident timeline

The Deployflow source frames audit-ready DevOps pipelines around automatic logging, immutable records, RBAC, encryption, monitoring, and preconfigured audit reports. It also emphasises that a DevOps pipeline should make changes repeatable, traceable, and auditable [14]. The BML EthSecDevOps source reinforces the same direction from an ethical perspective: ethics should be treated as a first-class concern, with early value assessment, stakeholder analysis, ethical code reviews, fairness testing, ethical deployment checklists, and continuous monitoring [13].

6.4 Policy-as-Code tools

Policy-as-Code tools such as Open Policy Agent (OPA), HashiCorp Sentinel, Chef InSpec, and Cloud Custodian can express compliance constraints in executable form. For example:

Listing 3: Illustrative OPA/Rego-style rule.

```
deny[msg] {
  input.resource.type == "database"
  not input.resource.encryption_at_rest
  msg := "Database must enable encryption at rest."
}
```

Listing 4: Illustrative retention rule.

```
deny[msg] {
  input.dataset.contains_personal_data
  input.dataset.retention_days > input.policy.max_retention_days
}
```

```
msg := "Personal-data retention exceeds the approved policy."
}
```

6.5 Active and passive responsibility in DevOps

Active responsibility becomes operational through blocking controls: the pipeline prevents non-compliant artefacts from reaching production. **Passive responsibility** becomes operational through evidence: every policy decision, exception, deployment, and runtime alert is logged for later audit and accountability.

This directly addresses the many-hands problem. Instead of relying on memory or informal coordination, the pipeline records decisions and makes responsibility attributable at the artefact level.

7 Expansion 3: MLOps and Runtime Assurance

7.1 Why AI requires a further expansion

AI systems add new sources of risk: training data may be biased, model behaviour may be statistically uncertain, the operational environment may diverge from the training environment, and performance may degrade after deployment. The responsible-AI software engineering roadmap highlights that responsible AI must be operationalised across governance, process, and system perspectives, not only at the algorithm level [8]. It also emphasises continuous monitoring, co-versioning of data/model/code/configuration, audit trails, and dynamic ethical risk assessment during operation [8].

How the diagram expands. The DevOps pipeline becomes an MLOps pipeline:

**Data → Training → Validation → Deployment → Runtime monitoring → Retraining
or rollback**

7.2 The W-shaped AI lifecycle

For AI systems, especially high-risk systems, the W-shaped lifecycle is more appropriate than a simple V-model. The TAID white paper describes the AI W-shape as an expansion of the traditional V-model that accounts for the specificities of AI constituents, including the tight connection between the AI model and its pre/post-processing [11]. It distinguishes engineering processes on the **host platform** from those on the **target platform** [11].

7.3 Design-time assurance

Design-time assurance is performed before deployment and focuses on whether the AI constituent satisfies its allocated requirements. It includes:

- dataset quality assessment and data lineage;
- bias and representativeness checks;

- model validation and independent test sets;
- robustness, stability, and generalisation analysis;
- explainability and interpretability assessment;
- conformity checks against legal and organisational policies;
- assurance cases linking claims, evidence, and residual risks.

This phase primarily supports active responsibility: the organisation must demonstrate that foreseeable harms were analysed and mitigated before deployment.

7.4 Runtime assurance

Runtime assurance is performed after deployment and focuses on whether the deployed system remains trustworthy under actual operating conditions. It includes:

- **data drift monitoring:** changes in input distributions;
- **concept drift monitoring:** changes in the relationship between inputs and outputs;
- **performance monitoring:** accuracy, latency, false positives/negatives, fairness metrics;
- **safety envelope monitoring:** detection of states outside the approved operational design domain;
- **fallback mechanisms:** rollback, safe mode, conservative decision logic;
- **human escalation:** routing uncertain or high-impact decisions to qualified humans.

Runtime assurance supports both active and passive responsibility. It actively prevents uncontrolled degradation and passively produces evidence for incident reconstruction.

7.5 Meaningful Human Control

Meaningful Human Control (MHC) is the mechanism that prevents technological autonomy from eliminating human accountability. The TAID white paper identifies MHC as a key open issue for trustworthy AI and links it to compliance, dignity, and responsibility [12]. In the pipeline, MHC should not be a vague statement. It must be engineered through concrete controls:

- explicit thresholds for human escalation;
- user interfaces that provide context, confidence, and explanation;
- authority to override or stop the system;
- logging of human decisions and machine recommendations;
- training and role definitions for operators.

7.6 Connection to superintelligence and value alignment

The report does not assume near-term superintelligence. However, the alignment lesson is still relevant: optimisation systems can pursue proxy objectives that diverge from human values. The pipeline therefore treats value alignment as an engineering problem: values are clarified, transformed into goals, operationalised as rules, enforced as policies, monitored at runtime, and periodically reassessed with stakeholders.

8 Integrated Final Pipeline

8.1 Final layered model

The final pipeline is not a single linear process. It is a layered architecture of artefacts and feedback loops.

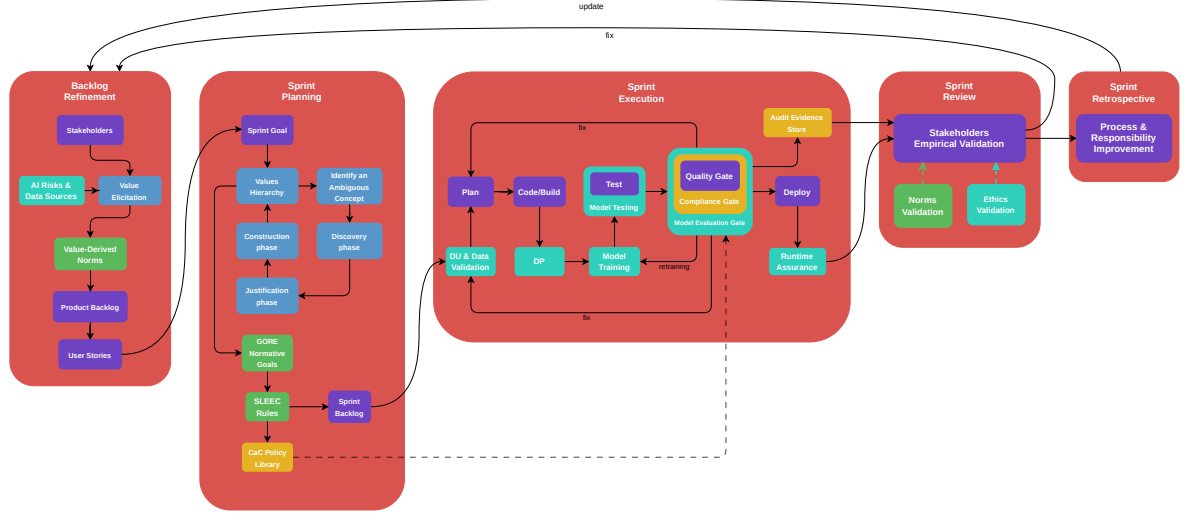


Figure 5: Final integrated pipeline. The complete model connects VSD, Conceptual Engineering, GORE/SLEEC, Compliance-as-Code, MLOps, evidence storage, model registry, explainability, and human oversight.

8.2 Traceability matrix

Table 4: End-to-end traceability from value to operational evidence.

Level	Artefact	Example	Evidence
Value	Value statement	Privacy must be pre-served	Stakeholder workshop notes
Concept	Clarified concept	Privacy as minimisation, control, confidentiality	Conceptual decision record
Norm	Normative statement	Only necessary data may be processed	Legal mapping to GDPR Art. 5/25
Goal	Normative goal	Minimise personal-data collection	GORE model

Rule	SLEEC rule	When unnecessary field is requested, block collection	Rule repository
Policy	Policy-as-code	Deny schema with unapproved PII	OPA/Sentinel decision log
Test	CI/CD test	Privacy regression test	CI report
Runtime	Monitor	Retention/drift/access alert	Immutable audit log
Governance	Review	Human approval or exception	Approval record and rationale

9 Mapping Course Topics to the Pipeline

Table 5: Connections with lecture topics.

Lecture topic	Pipeline connection
Cybernetics and behaviour	Agile, CI/CD, runtime monitoring, and human oversight are feedback loops that correct deviations from intended goals.
Machine intelligence	The pipeline treats AI as engineered decision-support systems whose outputs require assurance, not as moral agents.
Superintelligence and singularity	Value alignment is translated into a practical engineering sequence: values, goals, rules, policies, monitors.
Philosophy of mind	The Chinese Room motivates human review and semantic validation of formal rules.
AI ethics and machine ethics	The pipeline distinguishes ethics for creators from ethical behaviour constraints for machines.
Big data and algorithms	Bias, opacity, explainability, data lineage, and drift are treated as testable and monitorable properties.
Responsibility	Active responsibility maps to prevention; passive responsibility maps to auditability, incident reconstruction, and accountability.
Legal framework and CSR	GDPR, AI Act, CSR, and organisational governance become pipeline artefacts rather than external documents.

10 Implementation Blueprint

A practical implementation can proceed incrementally:

1. **Sprint 0:** define stakeholders, value sources, legal scope, and initial value hierarchy.

2. **Sprint 1:** convert the most important values into GORE normative goals and SLEEC-style rules.
3. **Sprint 2:** implement CI checks for the highest-risk GDPR controls: encryption, retention, access control, logging, and data minimisation.
4. **Sprint 3:** add policy-as-code gates with OPA or Sentinel.
5. **Sprint 4:** add audit evidence storage and dashboarding.
6. **Sprint 5:** for AI systems, add model registry, dataset lineage, model cards, fairness tests, and drift monitors.
7. **Sprint 6:** formalise human oversight, exception handling, and incident response procedures.

11 Conclusion

The final pipeline transforms ethics and legality from abstract external concerns into engineering artefacts. Agile provides the iterative structure; VSD introduces stakeholders and values; Conceptual Engineering clarifies the meaning of those values; GORE and SLEEC operationalise them as goals and rules; Compliance-as-Code enforces them in the CI/CD pipeline; MLOps and Runtime Assurance keep AI systems aligned after deployment.

The result is a pipeline oriented toward **socio-technical trustworthiness**. It does not guarantee perfect ethical behaviour, but it creates a disciplined framework for making values explicit, legal obligations traceable, compliance testable, runtime behaviour observable, and responsibility attributable.

References

- [1] N. Wiener, *Cybernetics: Or Control and Communication in the Animal and the Machine*, MIT Press, 1948.
- [2] A. M. Turing, “Computing Machinery and Intelligence,” *Mind*, vol. 59, no. 236, pp. 433–460, 1950. DOI: <https://doi.org/10.1093/mind/LIX.236.433>.
- [3] J. R. Searle, “Minds, Brains, and Programs,” *Behavioral and Brain Sciences*, vol. 3, no. 3, pp. 417–424, 1980. DOI: <https://doi.org/10.1017/S0140525X00005756>.
- [4] A. Matthias, “The responsibility gap: Ascribing responsibility for the actions of learning automata,” *Ethics and Information Technology*, vol. 6, pp. 175–183, 2004. DOI: <https://doi.org/10.1007/s10676-004-3422-1>.
- [5] S. Umbrello and O. Gambelin, *Agile as a Vehicle for Values: A Value Sensitive Design Toolkit*, preprint version, later published in Springer, 2023.
- [6] H. Veluwenkamp and J. van den Hoven, “Design for values and conceptual engineering,” *Ethics and Information Technology*, vol. 25, article 2, 2023. DOI: <https://doi.org/10.1007/s10676-022-09675-6>.
- [7] E. Silva Junior, L. Marsso, R. Caldas, M. Chechik, and G. N. Rodrigues, *Operationalizing Human Values in the Requirements Engineering Process of Ethics-Aware Autonomous Systems*, arXiv:2602.09921, 2026.
- [8] Q. Lu, L. Zhu, X. Xu, J. Whittle, and Z. Xing, *Towards a Roadmap on Software Engineering for Responsible AI*, arXiv:2203.08594, 2022.
- [9] P. Singh, “Integrating Compliance-as-Code into CI/CD Pipelines for Regulated Industries,” *International Journal of Innovative Research and Creative Technology*, vol. 11, no. 2, 2025.
- [10] L. Potharalanka, “Compliance-As-Code: A Framework for Regulatory Automation in Enterprise CI/CD Pipelines,” *Sarcouncil Journal of Engineering and Computer Sciences*, vol. 4, issue 7, pp. 257–267, 2025.
- [11] European Defence Agency, *Trustworthiness for AI in Defence: Developing Responsible, Ethical, and Trustworthy AI Systems for European Defence*, TAID White Paper, version 1.0, 09/05/2025.
- [12] European Defence Agency, *Trustworthiness for AI in Defence*, sections on Meaningful Human Control and Value-Based Engineering, 2025.
- [13] D. Webster and B. Johnson, “EthSecDevOps: Integrating Ethics as a First-Class Citizen in Product Development,” BML Global. Available at: <https://bml.global/ethsecdevops-integrating-ethics-as-a-first-class-citizen-in-product-development/>.
- [14] N. Ilic, “Making GDPR Practical: Real-World DevOps Pipelines That Pass Audits,” Deployflow, 2025. Available at: <https://deployflow.co/blog/making-gdpr-practical-devops-pipelines-pass-audits/>.

- [15] European Union, General Data Protection Regulation, Article 5, “Principles relating to processing of personal data.” Available at: <https://gdpr-info.eu/art-5-gdpr/>.
- [16] European Union, General Data Protection Regulation, Article 25, “Data protection by design and by default.” Available at: <https://gdpr-info.eu/art-25-gdpr/>.
- [17] European Data Protection Board, *Guidelines 4/2019 on Article 25 Data Protection by Design and by Default*, version 2.0, 2020.
- [18] European Commission, “AI Act,” Shaping Europe’s Digital Future. Available at: <https://digital-strategy.ec.europa.eu/en/policies/regulatory-framework-ai>.
- [19] European Union, Artificial Intelligence Act, Article 14, “Human oversight.” Available at: <https://artificialintelligenceact.eu/article/14/>.